

Collegium Witelona Uczelnia Państwowa w Legnicy
Wydział Nauk Technicznych i Ekonomicznych
Kierunek: Informatyka



Projekt z przedmiotu "Zaawansowane metody programowania"

Temat: "Qui La Carne".

Autorzy

Mateusz Bogacz-Drewniak, nr. indeksu: 44491

Mateusz Chimkowski, nr. indeksu: 43831

Szymon Mikołajek, nr. indeksu: 41105

Paweł Kruk, nr. indeksu: 43854

Miron Kwiatkowski, nr. indeksu: 42743

Prowadzący przedmiot
mgr inż. Krzysztof Rewak

Legnica, 2026

Spis treści

1	Opis funkcjonalny systemu	3
1.1	Wstęp	3
1.2	Główne funkcjonalności	3
1.2.1	Zarządzanie użytkownikami	3
1.2.2	Zarządzanie menu	3
1.2.3	Rezerwacje	4
1.2.4	Zarządzanie zamówieniami	4
1.2.5	Zarządzanie stolikami	4
1.2.6	System raportowania	5
1.2.7	Bezpieczeństwo i audyt	5
1.2.8	Komunikacja w czasie rzeczywistym	5
1.2.9	Integracje zewnętrzne	5
2	Opis technologiczny	6
2.1	Architektura ogólna	6
2.2	Stack technologiczny	6
2.2.1	Backend	6
2.2.2	Aplikacja Web	7
2.2.3	Aplikacja mobilna (Mateusz Chimkowski)	8
2.2.4	Aplikacja Mobilna (Miron Kwiatkowski)	9
2.2.5	Aplikacja Desktopowa	10
2.3	Architektura bazy danych	11
3	Interfejs aplikacji	12
3.1	Backend	12
3.2	Aplikacja Mobilna (Mateusz Chimkowski)	14
3.3	Aplikacja Mobilna (Miron Kwiatkowski)	15
3.4	Aplikacja Web	16
3.5	Aplikacja Desktop	17
4	Repozytoria kodu (GIT)	19
5	Instrukcja uruchomienia lokalnego	19
5.1	Backend	19
5.1.1	Wymagania wstępne	19
5.1.2	Instrukcja	19
5.2	Frontend	22
5.2.1	Wymagania wstępne	22
5.2.2	Instrukcja krok po kroku	22
6	Instrukcja uruchomienia zdalnego	22
6.1	Backend	22
6.1.1	Przygotowanie zewnętrznych usług	22
	Baza Danych (PostgreSQL):	23
	Magazyn Plików (S3 Storage):	23
6.2	Deployment na platformie Render.com	24

6.3	Frontend (Vercel)	25
7	Wzorce projektowe	27
7.1	Backend	27
7.2	Web	28
7.3	Aplikacja Mobilna (Mateusz Chimkowski)	29
7.4	Aplikacja mobilna (Miron Kwiatkowski)	30
7.5	Aplikacja Desktopowa	31
8	Wnioski projektowe	32
8.1	Mateusz Bogacz-Drewniak	32
8.2	Paweł Kruk	32
8.3	Mateusz Chimkowski	33
8.4	Szymon Mikołajek	34
8.5	Miron Kwiatkowski	35

1 Opis funkcjonalny systemu

1.1 Wstęp

Qui La Carne to nowoczesny system zarządzania restauracją oparty na architekturze REST API, umożliwiający kompleksowe zarządzanie operacjami restauracyjnymi. System wspiera obsługę rezerwacji stolików, zamawiania potraw, zarządzanie użytkownikami oraz komunikację w czasie rzeczywistym poprzez WebSocket.

1.2 Główne funkcjonalności

1.2.1 Zarządzanie użytkownikami

System obsługuje trzy role główne:

- **Manager (Admin)** – pełna kontrola nad systemem, zarządzanie pracownikami, daniami, stolami
- **Kelner (Waiter)** – przyjmowanie zamówień, zarządzanie rezerwacjami, obsługa stolów
- **Klient (Client)** – rezerwacja stolików, składanie zamówień, przeglądanie menu

Funkcjonalności użytkownika:

- Rejestracja i aktywacja konta
- Logowanie z obsługą 2FA (Google Authenticator)
- Logowanie przez Google OAuth2
- Zmiana hasła
- Reset hasła przez email
- Zarządzanie profilami pracowników (dla managera)
- Blokowanie/odblokowanie użytkowników
- Banowanie gości (systemy ban z przyczynami i datami)

1.2.2 Zarządzanie menu

System umożliwia:

- Przegląd pełnego menu restauracji z podziałem na kategorie
- Zarządzanie daniami (dodawanie, edycja, usuwanie)
- Definiowanie składników i alergenów
- Zarządzanie dostępnością dań
- Przesyłanie i przechowywanie zdjęć dań (S3 storage)
- Filtrowanie dań po kategoriach i dostępności
- Obsługa wielu języków (Polski, Angielski)

1.2.3 Rezerwacje

Funkcjonalności rezerwacji:

- Rezerwacja stolików przez klientów
- Zamawianie dań razem z rezerwacją
- Zarządzanie rezerwacjami przez kelnerów
- Śledzenie statusu rezerwacji (PENDING, IN_PROGRESS, NO_SHOW, COMPLETED)
- Przypisywanie kelnerów do rezerwacji
- Filtrowanie rezerwacji (po dacie, statusie, użytkowniku)
- Powiadomienia w czasie rzeczywistym o zmianach rezerwacji (WebSocket)

1.2.4 Zarządzanie zamówieniami

System obsługuje:

- Tworzenie zamówień dla stolów (z lub bez rezerwacji)
- Dodawanie pozycji do zamówienia
- Zarządzanie statusami pozycji (PENDING, PREPARING, READY, SERVED, CANCELLED)
- Zarządzanie statusami zamówienia (PENDING, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED)
- Śledzenie łącznej ceny zamówienia
- Notki do poszczególnych pozycji (wymogi dietetyczne, uwagi)
- Powiadomienia w czasie rzeczywistym dla kuchni
- Synchronizacja statusów zamówień na żywo (WebSocket)

1.2.5 Zarządzanie stolikami

Funkcjonalności:

- Definiowanie stolików z numerami i pojemnością
- Śledzenie statusu stołu (AVAILABLE, OCCUPIED, CLEANING, OUT_OF_SERVICE)
- Blokowanie/odblokowywanie stolów
- Rezerwacje stolików na wybrane godziny
- Powiadomienia o zmianach statusu w czasie rzeczywistym

1.2.6 System raportowania

System wspiera:

- Zgłaszanie problemów przez gości (GuestReports)
- Zarządzanie statusami raportów
- Podgląd raportów przez administrację
- Śledzenie odpowiadających pracowników

1.2.7 Bezpieczeństwo i audyt

- Autentykacja JWT z tokenami krótkotrwałymi
- Wymaganie 2FA dla wysoko uprzywilejowanych kont
- Google OAuth2 dla wygodnego logowania
- Ograniczenie przepustowości (rate limiting) przy użyciu Bucket4j
- CORS skonfigurowany dla bezpieczeństwa
- CSRF protection
- Audit logging - rejestracja wszystkich akcji użytkownika
- Soft delete (logiczne usuwanie) dla danych wrażliwych
- Banowanie użytkowników ze wskazaniem przyczyny

1.2.8 Komunikacja w czasie rzeczywistym

System WebSocket (STOMP) umożliwia:

- Powiadomienia o zmianach rezerwacji
- Powiadomienia o zmianach statusu zamówień
- Powiadomienia o zmianach dostępności stolów
- Event-driven architektura z kompletnym paylodem (nie wymagająca dodatkowych zapytań)
- Subskrypcje na temat (topic-based) dla elastycznego routingu wiadomości

1.2.9 Integracje zewnętrzne

- **Google OAuth2** – logowanie przez konto Google
- **AWS S3 (MinIO)** – przechowywanie zdjęć dań
- **Email (MailPit)** – wysyłanie emaili z potwierdzeniami, resetami hasła, aktywacją
- **Google 2FA** – dwuskładnikowa autentykacja
- **Redis** – cachowanie i rate limiting

2 Opis technologiczny

2.1 Architektura ogólna

System Qui La Carne jest oparty na nowoczesnej architekturze trójwarstwowej:

- **Warstwa prezentacji (Frontend)** – aplikacja webowa (React), mobilna (Kotlin i Dart) oraz dekstopowa (C#) (dostępne w osobnych repozytoriach)
- **Warstwa biznesowa (Backend)** – Spring Boot REST API
- **Warstwa danych** – PostgreSQL baza danych

2.2 Stack technologiczny

2.2.1 Backend

Obszar / Komponent	Technologia	Wersja	Przeznaczenie w projekcie
Język programowania	Java	21	Główny język programowania aplikacji.
Build Tool	Gradle	-	Narzędzie do budowania projektu i zarządzania zależnościami.
Framework główny	Spring Boot	3.5.13	Szkielet aplikacji, obsługa endpointów, bazy danych itp.
Zabezpieczenia	Spring Security	3.5.13*	Ochrona aplikacji, uwierzytelnianie i autoryzacja żądań HTTP.
Baza danych i Migracje	PostgreSQL & Flyway	18 / BOM*	Relacyjna baza danych oraz system wersjonowania i migracji schematów.
Cache i Rate Limiting	Redis & Bucket4j	BOM* & 8.10.1	Pamięć in-memory (cache) oraz mechanizm ograniczania ruchu.
Komunikacja Real-Time	Spring WebSockets	3.5.13*	Dwukierunkowa komunikacja w czasie rzeczywistym (STOMP).
Przechowywanie plików	AWS SDK (S3) / MinIO	2.42.23	Obsługa operacji na plikach z wykorzystaniem AWS SDK.
Logowanie (Tokeny)	JJWT	0.13.0	Zarządzanie, generowanie i walidacja tokenów JWT.
Uwierzytelnianie 2FA	Google Auth	1.5.0	Implementacja logowania dwuskładnikowego (TOTP).
Integracje zewnętrzne	Google API Client	2.7.2	Oficjalny klient do komunikacji z interfejsami API Google.
Kod / Mapowanie	Lombok & MapStruct	BOM* & 1.6.3	Automatyczna generacja kodu (np. gettery) i mapowanie obiektów.
Dokumentacja API	Springdoc (OpenAPI)	2.8.14	Generowanie interaktywnej dokumentacji dla API (Swagger UI).
*Wersje oznaczone jako BOM / 3.5.13 są zarządzane automatycznie przez Spring Boot.			

Tabela 1: Szczegółowy stack technologiczny backendu wraz z wersjami bibliotek.

2.2.2 Aplikacja Web

Obszar / Komponent	Technologia / Biblioteka	Wersja	Przeznaczenie w projekcie
Język programowania	TypeScript	5.9.2	Statycznie typowany nadzbiór języka JavaScript zwiększający czytelność i bezpieczeństwo kodu.
Biblioteka UI	React & React DOM	19.2.4	Główna biblioteka służąca do budowania interfejsu użytkownika z reużywalnych komponentów.
Framework i Routing	React Router	7.12.0	Kompleksowe zarządzanie nawigacją (routing) oraz obsługa renderowania po stronie serwera (SSR).
Build Tool / Bundler	Vite	7.1.7	Ultraszybkie środowisko deweloperskie i narzędzie do budowania zoptymalizowanej paczki aplikacji.
Stylizowanie (CSS)	Tailwind CSS	4.1.13	Utility-first CSS framework pozwalający na błyskawiczne i responsywne ostylowanie interfejsu.
Komunikacja Real-Time	@stomp/stompjs & sockjs-client	7.3.0 & 1.6.1	Obsługa połączeń WebSockets do dwukierunkowej, ciągłej komunikacji z backendem.
Autentykacja	Google OAuth (@react-oauth)	0.13.4	Gotowe komponenty umożliwiające bezpieczne logowanie do aplikacji za pomocą konta Google.
Wielojęzyczność (i18n)	i18next & react-i18next	25.8.17	System internacjonalizacji z wtyczką automatycznie wykrywającą preferowany język w przeglądarce.
Optymalizacja SEO / SSR	isbot	5.1.31	Narzędzie do wykrywania botów i crawlerów sieciowych, wspierające renderowanie dla wyszukiwarek.
Hosting i Bezpieczeństwo	Vercel (vercel.json)	-	Konfiguracja wdrożenia (przekierowania do index.html) oraz rygorystyczne nagłówki Content-Security-Policy (CSP).

Tabela 2: Szczegółowy stack technologiczny aplikacji webowej (frontendu) wraz z wersjami.

2.2.3 Aplikacja mobilna (Mateusz Chimkowski)

Obszar / Komponent	Technologia / Biblioteka	Wersja	Przeznaczenie w projekcie
Platforma Docelowa	Android SDK	Min: 24, Target: 35	Natywna platforma docelowa dla aplikacji mobilnej[cite: 12].
Język programowania	Kotlin (JVM 11)	2.0.21*	Główny język aplikacji, którego kompilator celuje w środowisko JVM 11[cite: 14].
Build Tool	Gradle (Kotlin DSL)	-	Narzędzie do budowania projektu, skonfigurowane przy użyciu skryptów KTS[cite: 11].
Interfejs użytkownika	Jetpack Compose	BOM**	Deklaratywny framework do budowy UI oparty na wytycznych Material 3[cite: 25].
Nawigacja wewnątrz aplikacji	Navigation Compose	2.8.5	Obsługa routingu i przejść pomiędzy ekranami w środowisku Jetpack Compose[cite: 25].
Lokalna baza danych	Room & SQLCipher	2.6.1 & 4.5.4	Biblioteki do lokalnego persystowania danych (Room) w pełni zaszyfrowanej bazie (SQLCipher)[cite: 25].
Komunikacja HTTP & API	Retrofit 2 & OkHttp 3	2.11.0 & 4.12.0	Zestaw do obsługi żądań sieciowych REST, komunikacji z backendem i logowania zapytań[cite: 25].
Serializacja danych	Kotlinx Serialization JSON	1.7.3	Oficjalna biblioteka Kotlin do wydajnej serializacji i deserializacji obiektów JSON[cite: 25].
Pobieranie obrazów (UI)	Coil Compose	2.5.0	Narzędzie do asynchronicznego ładowania i cache'owania obrazów bezpośrednio w UI[cite: 26].
Bezpieczeństwo (Tokeny)	Security Crypto	1.1.0	Służy do bezpiecznego przechowywania wrażliwych danych na urządzeniu (np. zaszyfrowane preferencje)[cite: 25].
Jakość kodu & Testy	Detekt & Kover	-	Statyczna analiza kodu i linter (Detekt) oraz narzędzie raportujące pokrycie kodu testami (Kover)[cite: 11].
*BOM - wersje zarządzane przez Bill of Materials Compose.			

Tabela 3: Szczegółowy stack technologiczny aplikacji mobilnej.

2.2.4 Aplikacja Mobilna (Miron Kwiatkowski)

Obszar / Komponent	Technologia / Biblioteka	Wersja	Przeznaczenie w projekcie
Platforma Docelowa	Flutter SDK	-	Wieloplatformowy framework UI użyty do budowy aplikacji mobilnej (offline-first).
Język programowania	Dart	≥3.0.0	Główny język programowania środowiska Flutter.
Zarządzanie stanem (State)	Flutter BLoC (Cubit)	8.1.3	Implementacja wzorca BLoC do reaktywnego zarządzania stanem interfejsu i logiką biznesową.
Lokalna baza danych	Isar	3.1.0+1	Szybka, lokalna baza danych NoSQL, stanowiąca fundament dla działania aplikacji w trybie offline-first.
Komunikacja HTTP & API	Dio	5.4.0	Rozbudowany klient HTTP do obsługi żądań sieciowych REST i komunikacji z backendem.
Komunikacja Real-time	WebSocket & Stomp*	3.0.3 & 3.0.1	Obsługa dwukierunkowej komunikacji w czasie rzeczywistym za pomocą protokołu STOMP.
Wstrzykiwanie zależności	GetIt & Injectable	7.6.4 & 2.3.2	Wzorzec Service Locator (GetIt) połączony z automatycznym generowaniem drzewa zależności.
Modele & Serializacja	Freezed & JSON Serializable	2.4.5 & 6.7.1	Automatyczne generowanie kodu dla niezmiennych modeli danych (immutable) oraz parsowanie JSON.
Programowanie funkcyjne	Dartz	0.10.1	Narzędzie wprowadzające koncepcje funkcyjne (np. klasę Either wykorzystywaną do obsługi błędów).
Bezpieczeństwo (Tokeny)	Flutter Secure Storage	9.0.0	Służy do bezpiecznego przechowywania wrażliwych danych na urządzeniu (np. zaszyfrowane tokeny uwierzytelniające).

Tabela 4: Szczegółowy stack technologiczny aplikacji mobilnej (Moduł Kelnera).

2.2.5 Aplikacja Desktopowa

Obszar / Komponent	Technologia / Biblioteka	Wersja	Przeznaczenie w projekcie
Platforma docelowa	.NET (Windows)	9.0	Główne środowisko uruchomieniowe dla aplikacji desktopowej.
Interfejs użytkownika	WPF (Windows Presentation Foundation)	-	Framework do tworzenia bogatego, natywnego interfejsu graficznego dla systemu Windows.
Wzorzec architektoniczny	CommunityToolkit.Mvvm	8.4.2	Ułatwienie implementacji wzorca MVVM (Model-View-ViewModel) oraz zarządzanie stanem.
Kontener DI / Cykl życia	Microsoft.Extensions.-Hosting	10.0.8	Konfiguracja wstrzykiwania zależności (Dependency Injection) i zarządzanie cyklem życia aplikacji.
ORM (Dostęp do danych)	Entity Framework Core (SQLite)	9.0.16	System mapowania obiektowo-relacyjnego służący do operacji na lokalnej bazie danych.
Lokalna baza danych (Szyfrowanie)	SQLCipher (SQLitePCLRaw)	1.1.14 / 2.1.11	Silnik lokalnej bazy danych wyposażony w mechanizm pełnego szyfrowania plików bazy.
Komunikacja Real-Time	Websocket.Client	5.5.0	Obsługa asynchronicznych połączeń WebSockets do dwukierunkowej komunikacji z serwerem.
Serializacja danych	Newtonsoft.Json	13.0.4	Wydajna i elastyczna biblioteka do serializacji oraz deserializacji obiektów z/do formatu JSON.

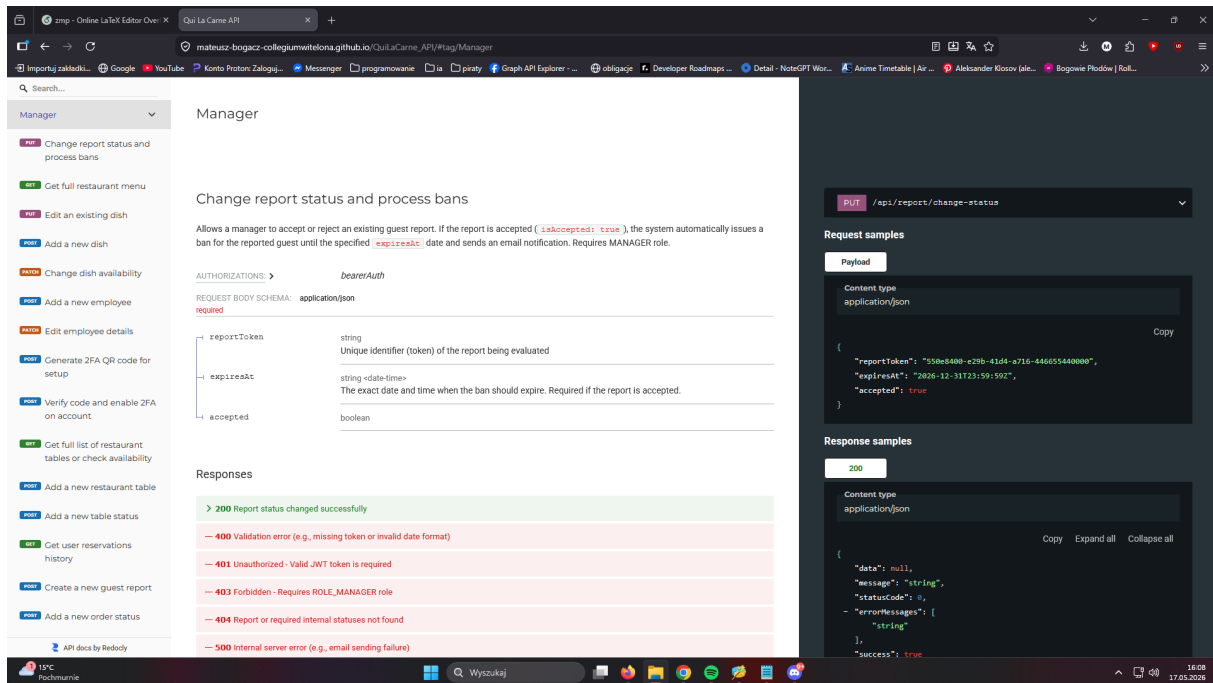
Tabela 5: Szczegółowy stack technologiczny aplikacji desktopowej.

Rysunek 1: Diagram związków encji (ERD) przedstawiający strukturę bazy danych systemu.

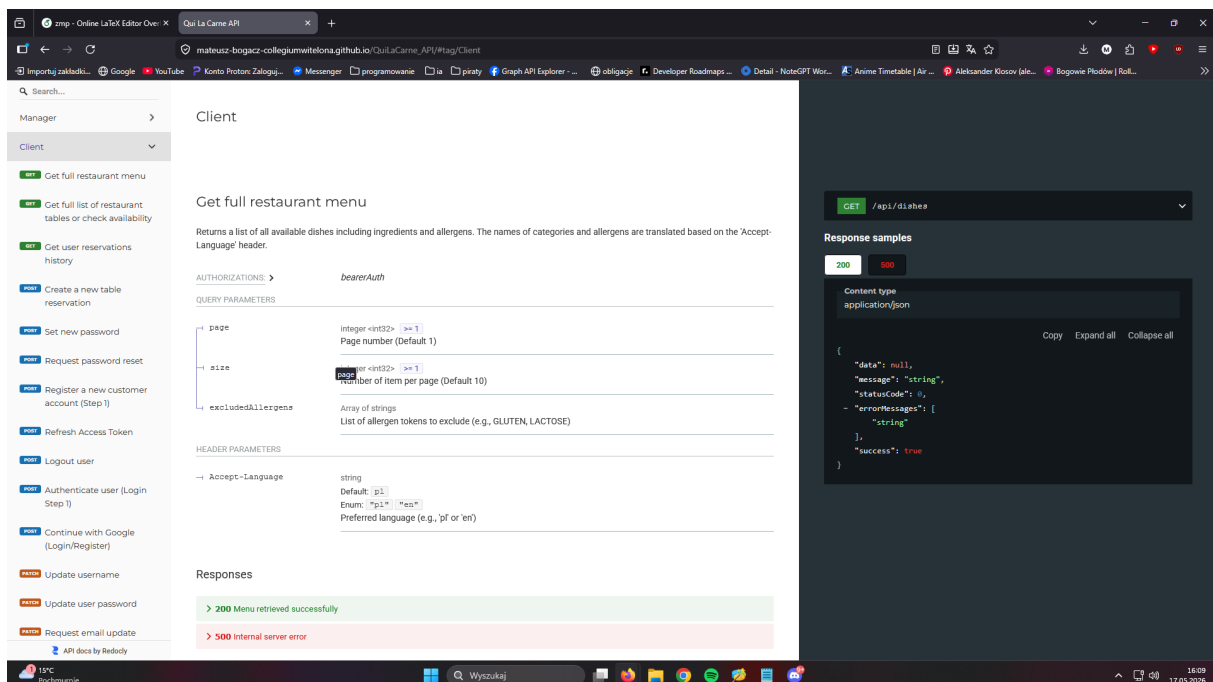


3 Interfejs aplikacji

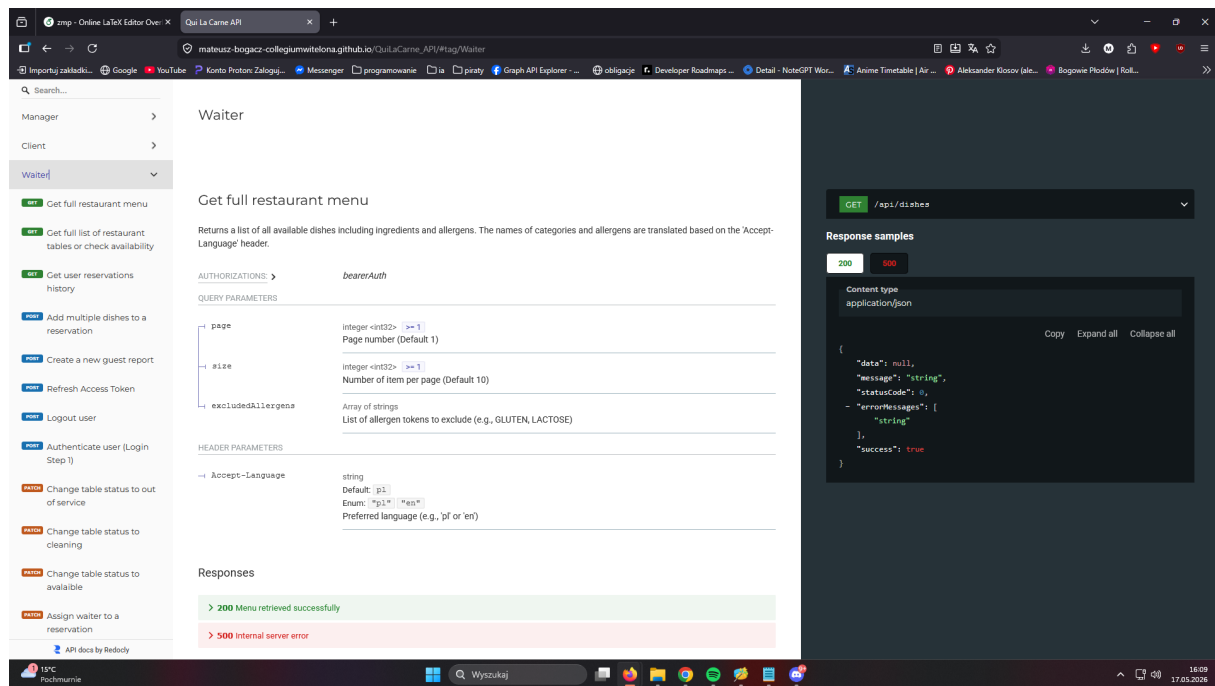
3.1 Backend



Rysunek 2: Przykładowy widok dokumentacji backend cz.1.

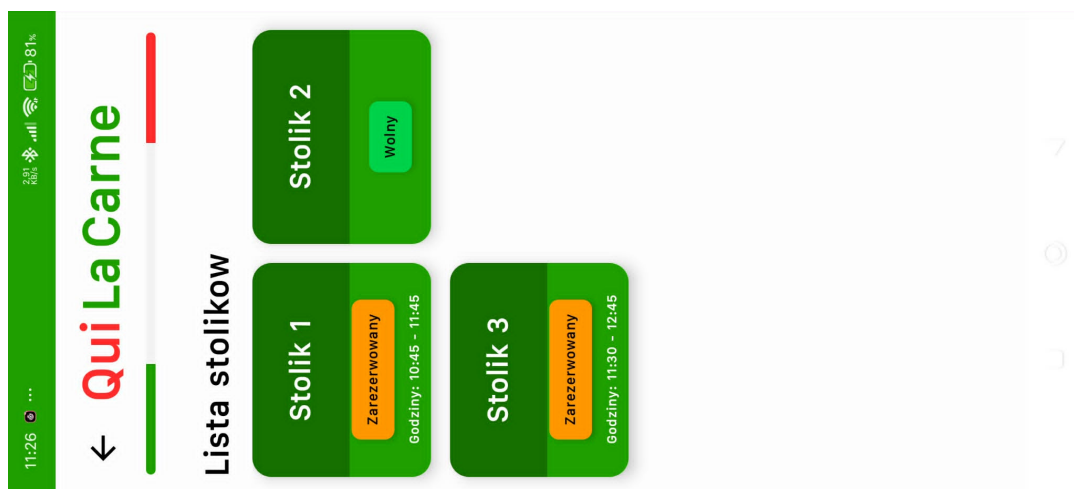


Rysunek 3: Przykładowy widok dokumentacji backend cz.2.

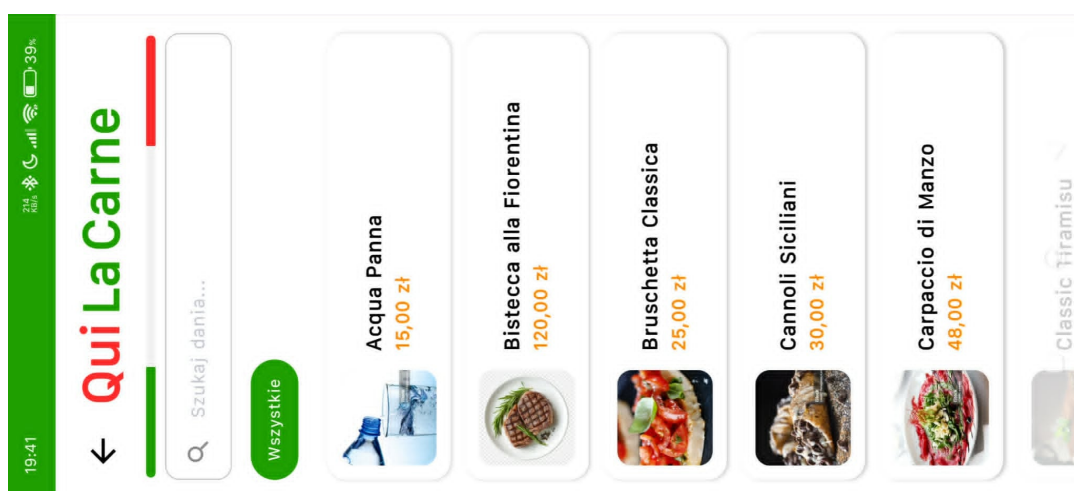


Rysunek 4: Przykładowy widok dokumentacji backend cz.3.

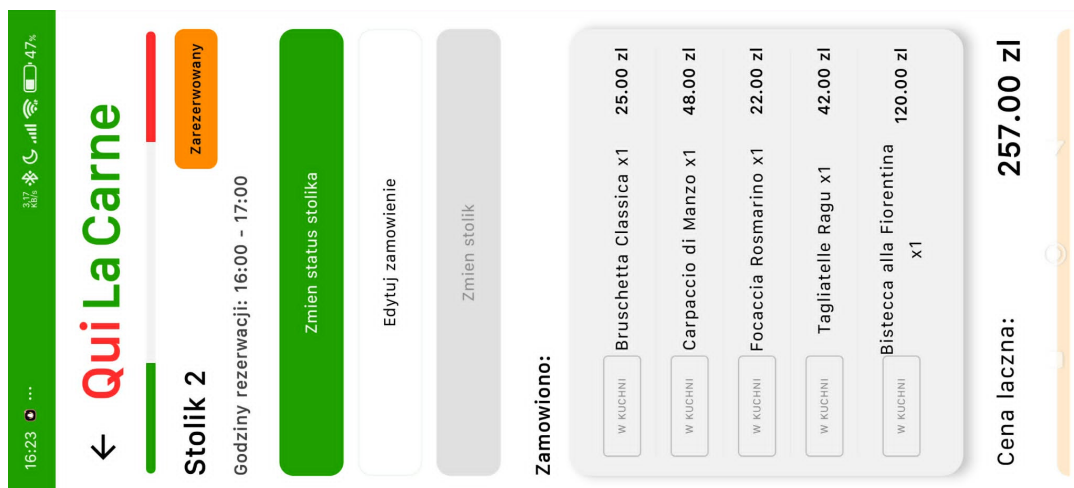
3.2 Aplikacja Mobilna (Mateusz Chimkowski)



Rysunek 5: Przykładowy widok z aplikacji mobilnej cz.1

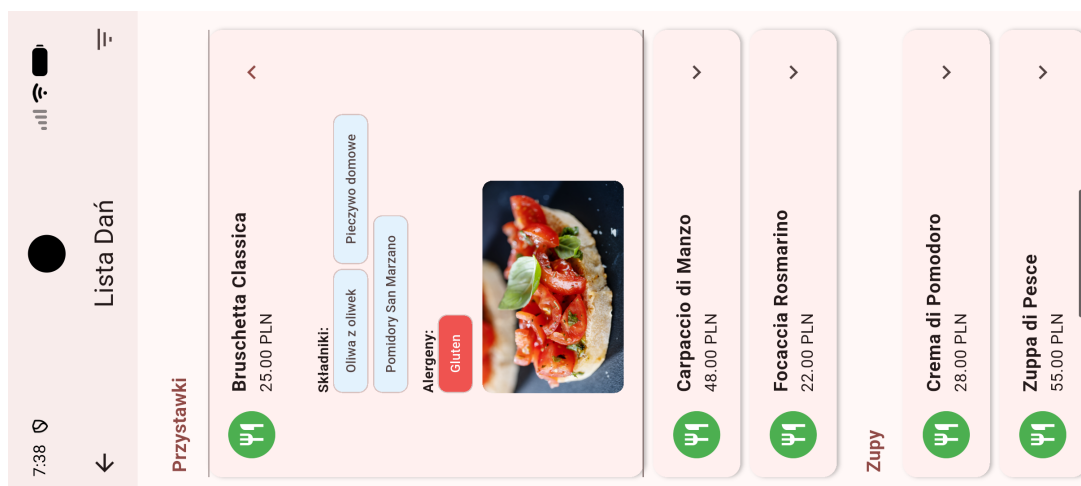


Rysunek 6: Przykładowy widok z aplikacji mobilnej cz.2

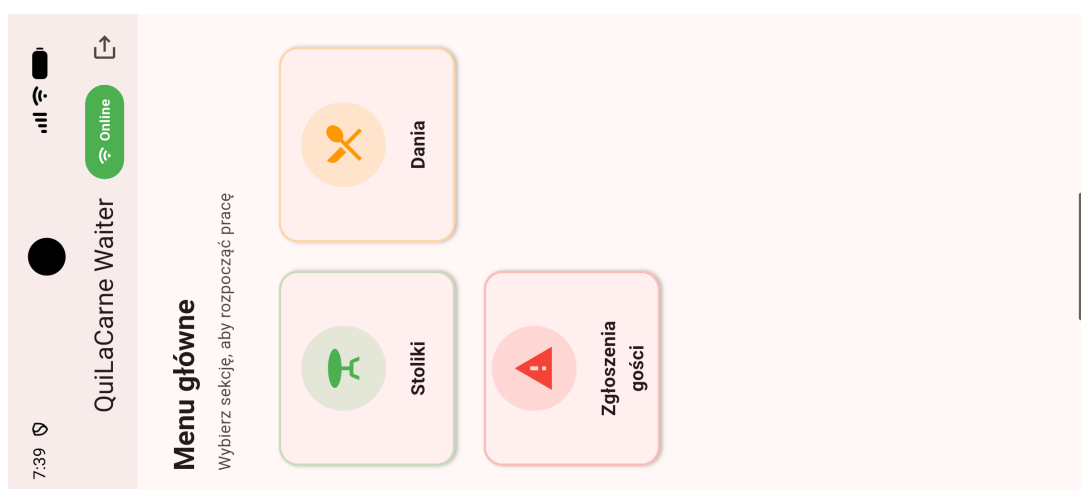


Rysunek 7: Przykładowy widok z aplikacji mobilnej cz.3

3.3 Aplikacja Mobilna (Miron Kwitatkowski)

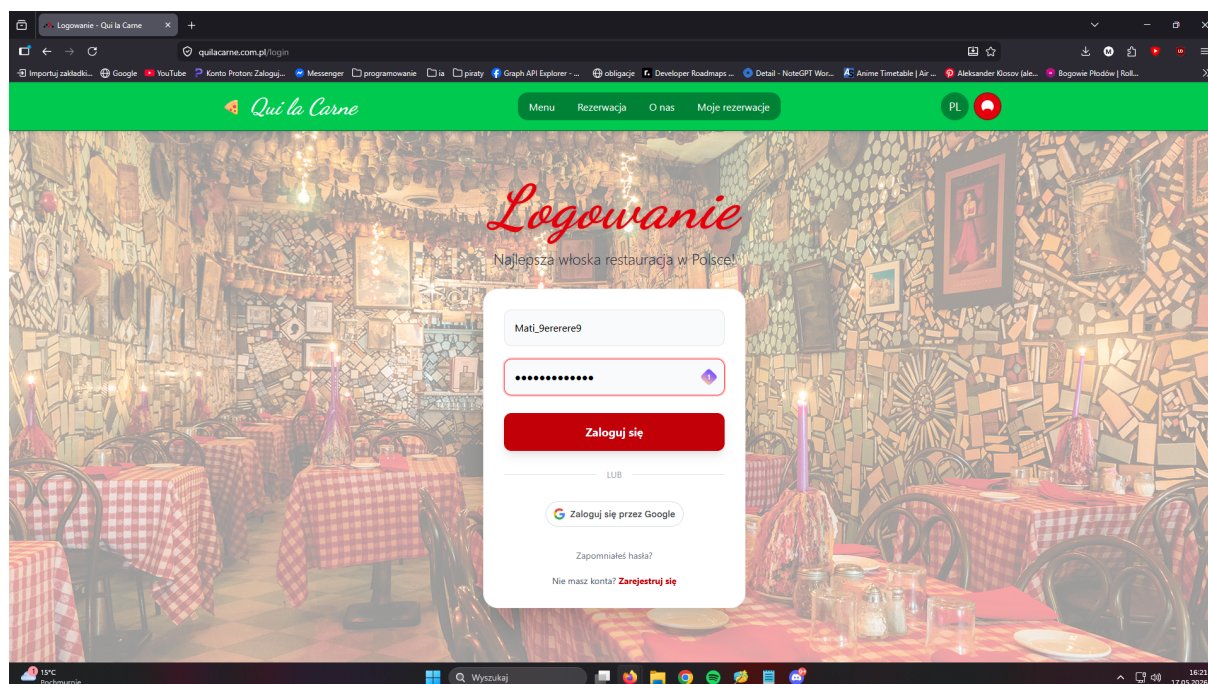


Rysunek 8: Przykładowy widok z aplikacji mobilnej cz.1 (Miron)

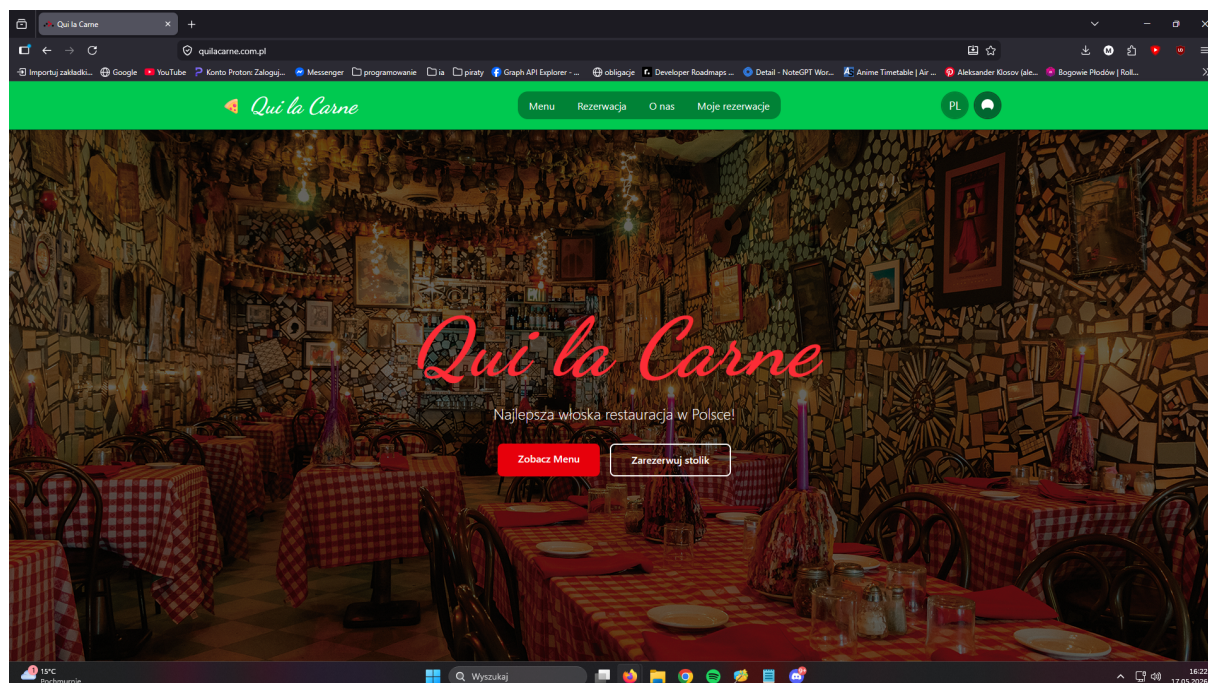


Rysunek 9: Przykładowy widok z aplikacji mobilnej cz.2 (Miron)

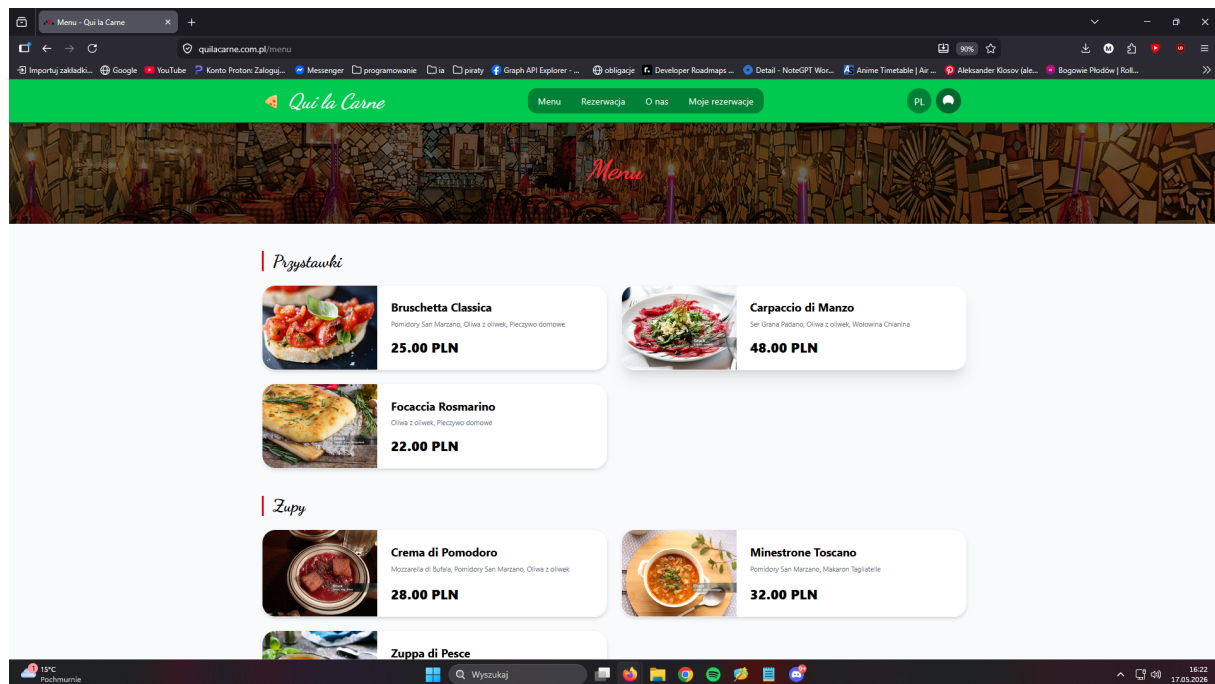
3.4 Aplikacja Web



Rysunek 10: Przykładowy widok aplikacji web cz.1.

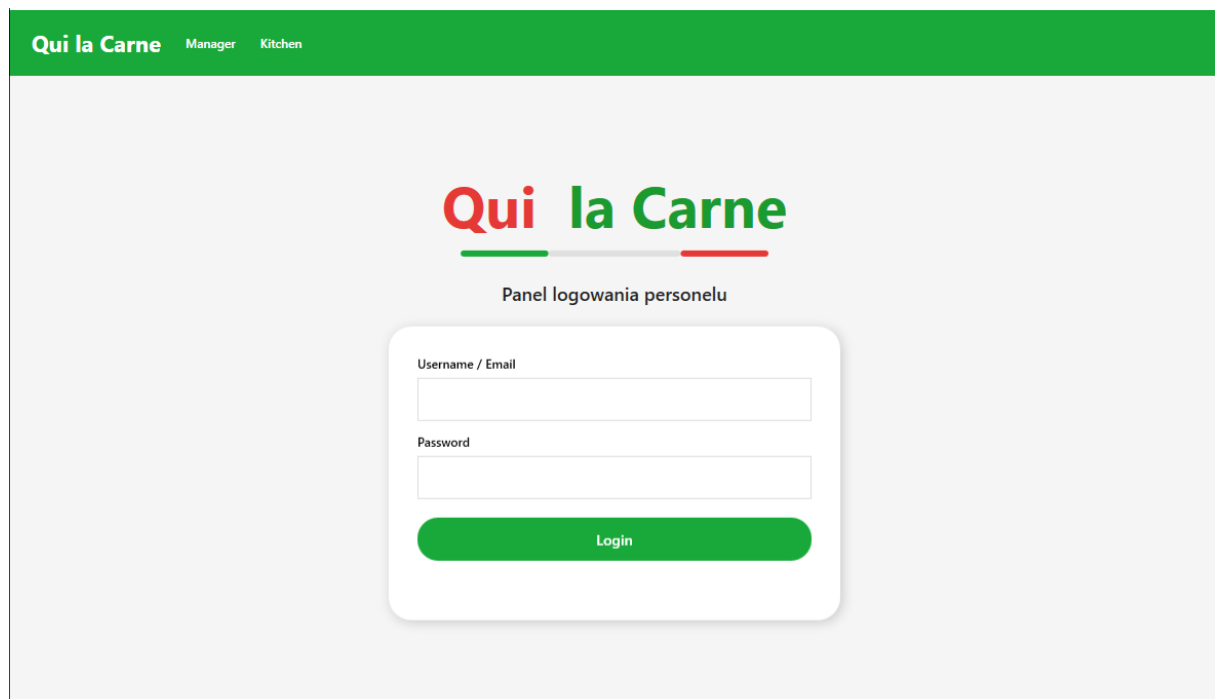


Rysunek 11: Przykładowy widok aplikacji web cz.2.

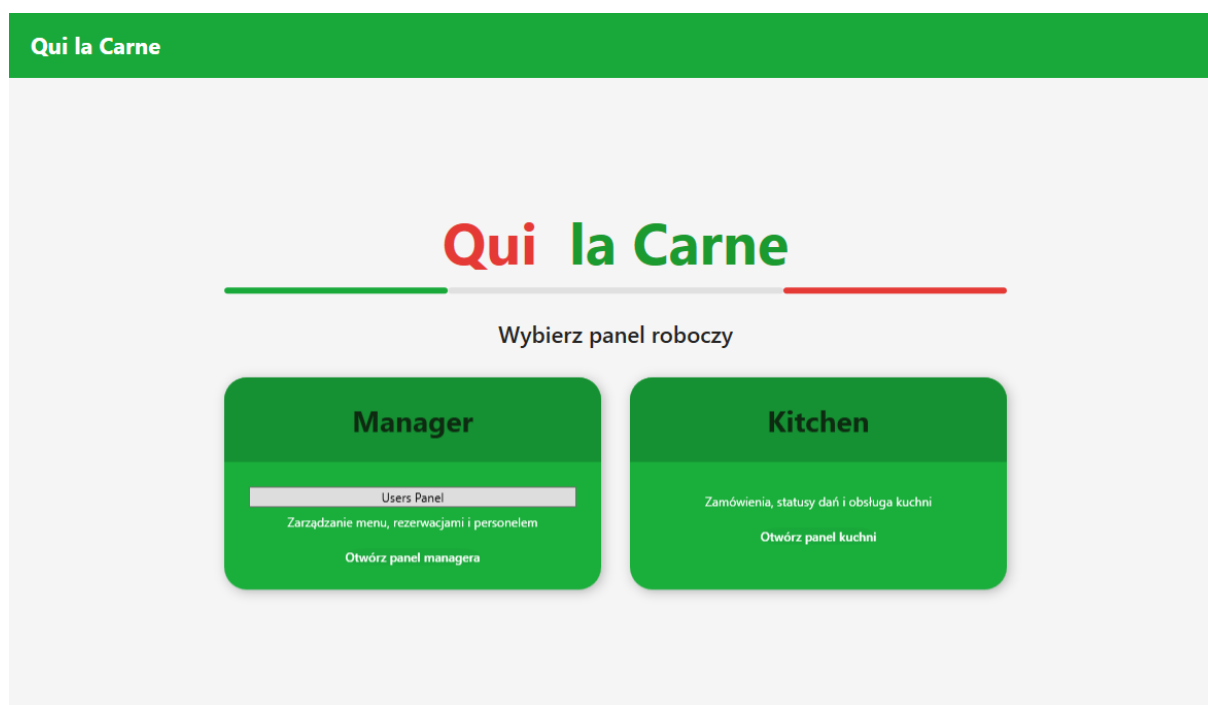


Rysunek 12: Przykładowy widok aplikacji web cz.3.

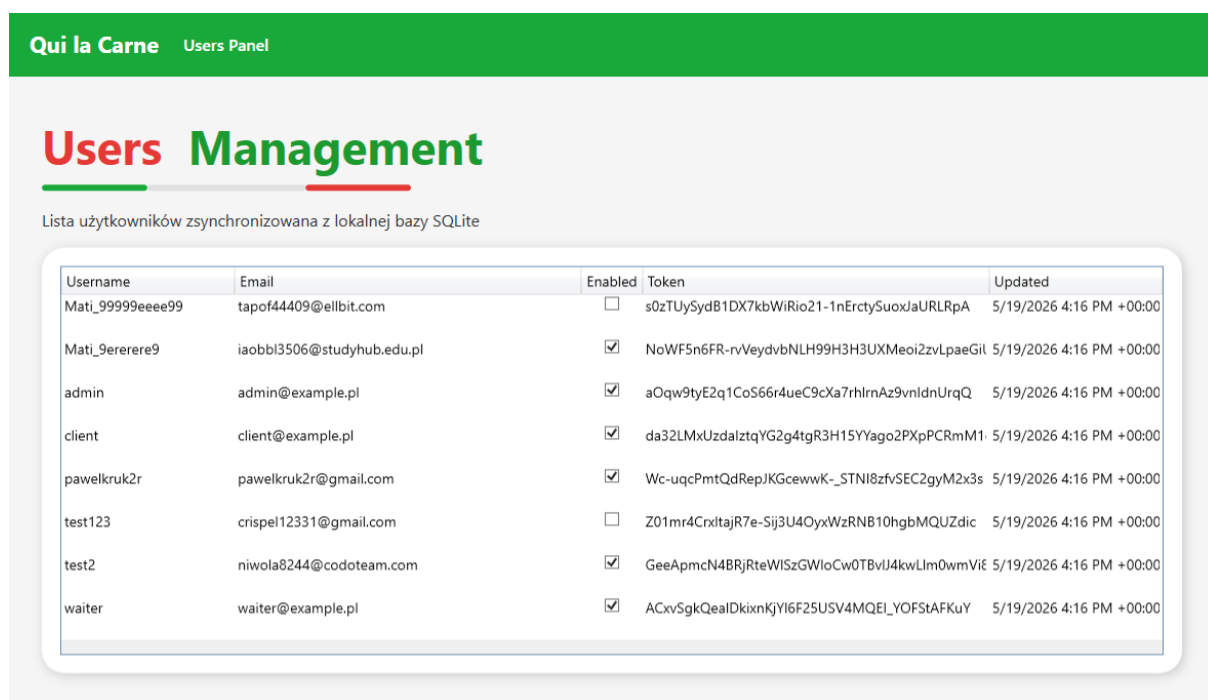
3.5 Aplikacja Desktop



Rysunek 13: Przykładowy widok aplikacji desktop cz.1.



Rysunek 14: Przykładowy widok aplikacji desktop cz.2.



Rysunek 15: Przykładowy widok aplikacji desktop cz.3.

4 Repozytoria kodu (GIT)

Zgodnie z założeniami architektury rozproszonej, system *Qui La Carne* został podzielony na osobne komponenty. Kod źródłowy każdej z aplikacji znajduje się w dedykowanych repozytoriach w serwisie GitHub:

- **Warstwa biznesowa (Backend REST API):**
https://github.com/mateusz-bogacz-collegiumwitelona/QuiLaCarne_API
- **Warstwa prezentacji (Aplikacja Webowa - React):**
<https://github.com/Kruk43854/web-ZMP-restauracja>
- **Warstwa prezentacji (Aplikacja Mobilna - Kotlin):**
<https://github.com/Chimek006/QuiLaCarne>
- **Warstwa prezentacji (Aplikacja Mobilna - Dart):**
<https://github.com/miron-kwiatkowski/ZmpQuiLaCarneDart>
- **Warstwa prezentacji (Aplikacja Desktopowa - C#):**
<https://github.com/SzymonMiko/CuiLaCarne-Desktop>

5 Instrukcja uruchomienia lokalnego

5.1 Backend

5.1.1 Wymagania wstępne

- Docker i Docker Compose (wersja 3.8+)
- Git
- Klucz JWT Secret (generator: <https://jwtsecrets.com/#generator>)
- Konto Google (dla OAuth2) – opcjonalnie
- Minimum 4GB RAM dla Docker

5.1.2 Instrukcja

1. Skopuj repozytorium lokalnie

```
git clone https://github.com/mateusz-bogacz-collegiumwitelona/QuiLaCarne_API
cd QuiLaCarne_API
```

Listing 1: Klonowanie repozytorium backend

2. Przygotowanie pliku .env

```
cp .env.example .env
```

Listing 2: Przygotowanie pliku .env backend

3. Edytuj plik `.env` i uzupełnij zmienne:

```
# Database
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres_password
POSTGRES_DB=quilacarne

# JWT
API_JWT_SECRET_KEY=YOUR_GENERATED_SECRET_KEY_HERE
API_JWT_EXPIRATION=3600000

# Frontend URL (CORS)
FRONTEND_URL=http://localhost:5173

# Redis
REDIS_HOST=redis
REDIS_PORT=6379

# S3/MinIO
S3_ENDPOINT=http://minio:9000
S3_PUBLIC_ENDPOINT=http://localhost:9000
S3_REGION=us-east-1
S3_ACCESS_LOGIN=minioadmin
S3_SECRET_PASSWORD=minioadmin
S3_BUCKET_NAME=quilacarne

# Google OAuth2 (opcjonalnie)
GOOGLE_CLIENT_ID=your_google_client_id

# Logging
API_LOGGING_LEVEL=DEBUG
API_SHOW_SQL=false
API_FORMAT_SQL=false
```

Listing 3: Przykładowy `.env` dla backend

4. Uruchomienie Docker Compose

```
docker compose up --build
```

Listing 4: Uruchomienie lokalne backend

Ten proces:

- Zbuduje obraz Docker dla aplikacji
- Uruchomi PostgreSQL, Redis, MailPit i MinIO
- Uruchomi Spring Boot aplikację
- Wykonamy migracje Flyway

Aplikacja będzie dostępna na:

- API: **http://localhost:8080**
- Swagger UI: **http://localhost:8080/swagger-ui/index.html**
- MailPit: **http://localhost:8025**
- MinIO: **http://localhost:9001**
- WebSocket: **ws://localhost:8080/ws-qlc**

5. Tworzenie konta administratora (opcjonalnie)

Bez dostępu do frontendu, utwórz konto admina poprzez CLI:

```
docker compose run --rm -it api --create-admin
```

Listing 5: Uruchomienie kreatora admina backend

System poprosi o:

- Username
- Email
- Hasło

Po utworzeniu, ponownie uruchom kontener:

```
docker compose down  
docker compose up
```

Listing 6: Restart kontenera backend

6. Krok 5: Uruchomienie testów (opcjonalnie)

```
docker compose -f compose.test.yml run --rm test
```

Listing 7: Uruchomienie testów backend

5.2 Frontend

5.2.1 Wymagania wstępne

- Zainstalowane środowisko **Node.js** (wersja 18.x lub nowsza).
- Menedżer pakietów **npm** (instalowany domyślnie wraz z Node.js).
- Uruchomiony lokalny serwer backendowy (Spring Boot) w celu zapewnienia pełnej funkcjonalności logiki aplikacji.

5.2.2 Instrukcja krok po kroku

1. Sklonowanie repozytorium

Pobierz kod źródłowy projektu na swój komputer lokalny za pomocą systemu kontroli wersji Git:

```
git clone https://github.com/Kruk43854/web-ZMP-restauracja
cd web-ZMP-restauracja
```

2. Instalacja zależności

Pobierz i zainstaluj wszystkie niezbędne pakiety zewnętrzne (w tym React, Vite, TailwindCSS) zdefiniowane w pliku `package.json`:

```
npm install
```

3. Konfiguracja zmiennych środowiskowych

W głównym katalogu projektu utwórz plik o nazwie `.env` i skonfiguruj w nim adres URL wskazujący na Twój lokalny serwer backendowy:

```
VITE_API_URL=http://localhost:8080
```

4. Uruchomienie aplikacji

Wystartuj lokalny serwer deweloperski narzędzia Vite:

```
npm run dev
```

Aplikacja będzie od tego momentu dostępna w przeglądarce, domyślnie pod adresem `http://localhost:5173`.

6 Instrukcja uruchomienia zdalnego

6.1 Backend

6.1.1 Przygotowanie zewnętrznych usług

Zanim przystąpisz do wdrożenia aplikacji na platformie Render, musisz utworzyć konta i wygenerować dane dostępowe w poniższych serwisach.

1. Supabase (Baza danych PostgreSQL i S3 Storage)
Supabase służy jako baza relacyjna oraz jako magazyn plików kompatybilny z AWS S3.

Baza Danych (PostgreSQL):

- Zaloguj się na panel Supabase.com.
- Kliknij **New Project** i uzupełnij dane (np. `qui-la-carne-db`).
- Wybierz region najbliższy Twoim użytkownikom (np. *Frankfurt*) i ustaw **silne hasło do bazy danych**. Zapisz je!
- Po utworzeniu projektu przejdź do: **Project Settings** (ikonka zębatki na dole po lewej) → **Database**.
- Znajdź sekcję **Connection string**, wybierz zakładkę **URI** i skopiuj podany link. Będzie on wyglądał następująco:

```
region postgresql://postgres.[ID_PROJEKTU]:[TWOJE_HASLO]@aws-0-  
.pooler.supabase.com:6543/postgres
```

*(Pamiętaj, aby podmienić [TWOJE_HASLO] na hasło ustawione w punkcie 3!) –
Zapisz ten link jako POSTGRES_CONNECTION_STRING.*

Magazyn Plików (S3 Storage):

- (a) W panelu Supabase przejdź do zakładki **Storage** (ikonka wiadra w menu bocznym).
- (b) Kliknij **New Bucket**, nadaj mu nazwę (np. quillacarne-media) i zaznacz opcję **Public**.
- (c) Przejdź do **Project Settings** → **Storage**.
- (d) Z sekcji **S3 Connection** skopiuj następujące dane:
 - **Endpoint** (np. `https://[ID_PROJEKTU].supabase.co/storage/v1/s3`)
 - **Region** (np. `eu-central-1`)
- (e) Kliknij przycisk **New Access Key**, wprowadź dowolną nazwę i skopiuj wygenerowane parametry: **Access Key ID** oraz **Secret Access Key** (ten drugi wyświetli się tylko raz!).

2. Redis Cloud (Obsługa Cache)

- (a) Zarejestruj się na platformie Redis Cloud.
- (b) Utwórz darmową subskrypcję (*Fixed plan* → *Free 30MB*).
- (c) Wybierz dostawcę chmury (np. AWS) oraz region (np. *Frankfurt / eu-central-1*).
- (d) Po utworzeniu bazy wejdź w jej szczegóły i zapisz:
 - **Public endpoint**: Adres w formacie `redis-12345.c241.eu-central-1-1.ec2.redislabs.com:12345`. Rozdziel go na **host** (tekst przed dwukropkiem) oraz **port** (liczba po dwukropku).
 - **Security** → **Default user password**: Skopiuj wygenerowane hasło.

3. Brevo (Wysyłka wiadomości SMTP)

- (a) Zaloguj się na swoje konto w serwisie Brevo.
- (b) Kliknij nazwę profilu w prawym górnym rogu i wybierz **SMTP & API**.
- (c) Przejdź do zakładki **SMTP**. Zapisz widoczny tam **Login** (Twój adres e-mail).
- (d) Kliknij **Generate a new SMTP key**, nazwij go (np. *Render-API*) i skopiuj wygenerowaną wartość (będzie to hasło SMTP).

6.2 Deployment na platformie Render.com

Render uruchomi aplikację na podstawie pliku `Dockerfile` znajdującego się w Twoim repozytorium.

1. Zaloguj się na Render.com.
2. Kliknij niebieski przycisk **New** i wybierz opcję **Web Service**.
3. Połącz swoje konto z GitHubem/GitLabem i wskaż repozytorium z kodem źródłowym aplikacji.
4. W sekcji konfiguracyjnej ustaw:
 - **Name**: `quilacarne-api`
 - **Region**: Wybierz region w Europie (np. *Frankfurt*), aby zminimalizować opóźnienia do bazy danych i aplikacji Redis.
 - **Branch**: `main` (lub inny produkcyjny).
 - **Runtime**: Wybierz **Docker**.
 - **Instance Type**: Wybierz plan **Free**.

5. Konfiguracja Zmiennych Środowiskowych

Na samym dole strony konfiguracyjnej usługi w Renderze przejdź do sekcji **Environment Variables**. Klikaj przycisk **Add Environment Variable** i wprowadź klucze oraz wartości zgodnie z poniższą specyfikacją:

```
# Spring Boot
API_FORMAT_SQL=false
API_LOGGING_LEVEL=DEBUG
API_PORT=8080
API_SHOW_SQL=false
JWT_SECRET_KEY= #wygenerowany wcześniej JWT secret

FRONTEND_URL= #url fornendu web

GOOGLE_CLIENT_ID= #client id z google cloude

# Postgres SQL
POSTGRES_HOST=#host z superbase
SPRING_DATASOURCE_PASSWORD= #hasło dostępu do projektu z superbase
SPRING_DATASOURCE_URL= #url bazy danych z superbase
SPRING_DATASOURCE_USERNAME= #nazwa użytkownika w bazie danych z superbase

# S3
S3_ACCESS_LOGIN=#login z superbase
S3_BUCKET_NAME= #nazwa bucketa ustawionego w superbase
S3_ENDPOINT= #link dosępowy z panelu superbase
S3_PUBLIC_ENDPOINT= #link dosępowy z panelu superbase
S3_REGION=#region z superbase
S3_SECRET_PASSWORD= #hasło dostępu do projektu z superbase

# REDIS
SPRING_DATA_REDIS_HOST= #host z rediscloude
SPRING_DATA_REDIS_PASSWORD= #hasło do projektu rediscloude
SPRING_DATA_REDIS_PORT= #port z rediscloude
SPRING_DATA_REDIS_SSL=false

# Mailing
API_EMAIL_FROM= #skład idzie mail (to co się ustawiło na brevo)
SPRING_EMAIL_USERNAME= #skład idzie mail (to co się ustawiło na brevo)
SPRING_MAIL_HOST= #host brevo
SPRING_MAIL_PASSWORD= #klucz z brebo
SPRING_MAIL_PORT= #port z brevo
SPRING_MAIL_USERNAME= #username z brevo
```

Listing 8: Konfiguracja zmiennych środowiskowych w Render

6.3 Frontend (Vercel)

Aplikacja frontendowa zbudowana w oparciu o narzędzie Vite jest w pełni zoptymalizowana do automatycznego wdrożenia w chmurze Vercel przy użyciu mechanizmów ciągłej integracji i wdrażania (CI/CD) bezpośrednio z repozytorium GitHub.

Instrukcja krok po kroku

1. Połączenie z repozytorium

- Zaloguj się na platformie Vercel (zalecana autoryzacja poprzez konto GitHub).
- W panelu głównym dashboardu kliknij przycisk **"Add New..."** i wybierz **"Project"**.
- Znajdź na liście repozytoriów pozycję **web-ZMP-restauracja** i kliknij przycisk **"Import"**.

2. Konfiguracja budowania (Build Settings)

Vercel automatycznie wykryje środowisko produkcyjne Vite. Upewnij się, że podstawowe parametry konfiguracji są ustawione następująco:

- **Framework Preset:** Vite
- **Build Command:** `npm run build`
- **Output Directory:** `dist`
- **Install Command:** `npm install`

3. Zmienne środowiskowe (Environment Variables)

Rozwiń sekcję "**Environment Variables**" i dodaj zmienną środowiskową niezbędną do komunikacji z produkcyjnym serwerem API:

- **Key:** `VITE_API_URL`
- **Value:** `<ADRES_TWOJEGO_PRODUKCYJNEGO_BACKENDU>` (np. `https://api.quilacarne.com.pl`)
- Kliknij przycisk "**Add**".

4. Inicjalizacja wdrożenia (Deploy)

- Kliknij przycisk "**Deploy**".
- Platforma Vercel rozpocznie proces alokacji zasobów, instalacji pakietów oraz budowania zoptymalizowanej wersji produkcyjnej.
- Po zakończeniu procesu aplikacja zostanie opublikowana pod darmową subdomeną Vercel, którą można później zastąpić własną domeną w zakładce *Settings* → *Domains*.

5. Ciągła Integracja (CI/CD)

Proces wdrażania został w pełni zautomatyzowany. Od tego momentu każdy nowy kod wypchnięty komendą `git push` na główną gałąź (`main` lub `master`) automatycznie wyzwoi nowy proces budowania w chmurze Vercel, aktualizując aplikację produkcyjną bez konieczności ręcznej interwencji.

7 Wzorce projektowe

7.1 Backend

Tabela obejmuje 12 klasycznych wzorców projektowych użytych w projekcie.

Kategoria	Wzorzec	Użyty	Gdzie / uwaga
Kreacyjne	Singleton	Tak	Domyślny scope beanów Spring to singleton; dotyczy m.in. klas <code>@Service</code> , <code>@Repository</code> , <code>@Component</code> .
Kreacyjne	Factory Method	Tak	Metody <code>@Bean</code> w <code>config/ApplicationConfig.java</code> i <code>config/SecurityConfig.java</code> tworzą i udostępniają obiekty.
Kreacyjne	Abstract Factory	Tak	Kontener Spring (<code>ApplicationContext</code> / <code>BeanFactory</code>) dostarcza rodziny powiązanych beanów infrastrukturalnych.
Kreacyjne	Builder	Tak	<code>helpers/WebSocketEvent.java</code> : adnotacja <code>@Builder</code> i wywołania <code>WebSocketEvent.builder()</code> .
Strukturalne	Facade	Tak	<code>facade/*Facade.java</code> to przykład fasady powstały po podziale dużych serwisów na mniejsze.
Strukturalne	Proxy	Tak	Spring AOP tworzy proxy m.in. dla adnotacji <code>@Transactional</code> , <code>@Cacheable</code> , <code>@CacheEvict</code> w klasach serwisowych.
Behawioralne	Chain of Responsibility	Tak	Łańcuch filtrów bezpieczeństwa Spring Security, m.in. <code>config/JwtAuthenticationFilter.java</code> .
Behawioralne	Command	Tak	<code>commands/AdminCommands.java</code> implementuje <code>CommandLineRunner</code> , czyli obiekt polecenia uruchamiany przez framework.
Behawioralne	Observer	Tak	WebSocket pub/sub: <code>services/NotificationServices.java</code> wysyła zdarzenia do <code>/topic/...</code> .
Behawioralne	State	Tak	<code>ReservationStateLogic</code> jest odpowiedzialny za zmianę stanów Rezerwacji.
Behawioralne	Strategy	Tak	Interfejsy <code>DishSearchStrategy</code> czy <code>ReservationCreateValidator</code> realizują wybór strategii implementacji.
Behawioralne	Template Method	Tak	<code>config/JwtAuthenticationFilter.java</code> nadpisuje <code>doFilterInternal()</code> z <code>OncePerRequestFilter</code> .

Tabela 6: Tabela użytych wzorców projektowych na backendzie

Uwaga metodologiczna: za *użyty* uznano zarówno wzorce zaimplementowane bezpośrednio w kodzie projektu, jak i wzorce realizowane przez framework (Spring), jeżeli są faktycznie wykorzystywane w tej aplikacji.

7.2 Web

Kategoria	Wzorzec	Użyty	Gdzie / uwaga
Kreacyjne	Singleton	Tak	<code>AuthContext.tsx</code> (<code>AuthProvider</code>) służy jako jedno, scentralizowane źródło prawdy (Single Source of Truth) o stanie sesji dla całej aplikacji. Podobnie <code>useWebSocket.ts</code> utrzymuje pojedynczą instancję klienta <code>WebSocket</code> .
Strukturalne	Facade	Tak	Hook <code>useWebSocket.ts</code> ukrywa skomplikowaną konfigurację <code>SockJS</code> i <code>@stomp/stompjs</code> . Podobnie <code>authFetch</code> (w <code>AuthContext.tsx</code>) upraszcza natywną metodę <code>fetch</code> , automatycznie dołączając nagłówki i obsługując CSRF.
Strukturalne	Proxy	Tak	Metoda <code>authFetch</code> działa jako Proxy Ochronne (Protection Proxy) – przechwytuje i sprawdza odpowiedzi z serwera. Jeśli wykryje błąd 401/403, automatycznie zamyka sesję (czyści <code>localStorage</code> i wylogowuje).
Behawioralne	Observer	Tak	Mechanizm subskrypcji <code>WebSockets</code> . Komponent <code>menu.tsx</code> nasłuchuje zmian pod adresami <code>/topic/menu/dishes</code> i <code>/topic/menu/availability</code> , odświeżając dane po otrzymaniu wiadomości.
Behawioralne	Template Method	Tak	Logika w <code>authFetch</code> definiuje stały szkielet każdego autoryzowanego żądania HTTP (ustawianie <code>Content-Type</code> , wydobywanie i dodawanie <code>X-XSRF-TOKEN</code>), pozwalając komponentom określić tylko zmienne elementy (URL, payload).
Behawioralne	Strategy	Tak	Dynamiczne filtrowanie dań w <code>menu.tsx</code> na podstawie tablic <code>selectedCategories</code> i <code>excludedAllergens</code> . W locie wylicza <code>filteredMenu</code> , zmieniając strategię wyświetlania w zależności od wprowadzonych przez użytkownika filtrów.

Tabela 7: Wzorce projektowe w warstwie frontendu (React).

7.3 Aplikacja Mobilna (Mateusz Chimkowski)

Wzorzec projektowy	Kategoria	Przykład w projekcie	Zastosowanie w projekcie
Builder	Kreacyjny	Room.databaseBuilder, OkHttpClient.Builder, Retrofit.Builder	Pozwala tworzyć złożone obiekty krok po kroku, np. bazę Room z migracjami i szyfrowaniem SQLCipher albo klienta HTTP z interceptorami i authenticatorem.
Factory Method / Factory	Kreacyjny	createRetrofit(), Retrofit.create(), AppDatabase.getDatabase()	Ukrywa szczegóły tworzenia obiektów, takich jak serwisy API lub instancja bazy danych. Kod korzystający z tych komponentów nie musi znać pełnej konfiguracji ich tworzenia.
Facade	Strukturalny	SyncRepository, RetrofitClient	Upraszcza dostęp do złożonej logiki aplikacji. Repozytorium ukrywa szczegóły komunikacji z API, lokalnej bazy danych, synchronizacji, tokenów oraz cache'owania danych.
Proxy	Strukturalny	Serwisy Retrofit, np. AuthService, TableService, OrderService	Interfejsy Retrofit działają jako pośrednicy między aplikacją a zdalnym API. Kod wywołuje metody serwisu tak, jakby były lokalne, a Retrofit zamienia je na żądania HTTP.
Observer	Behawioralny	Flow, StateFlow, combine, DAO Room	Pozwala ekranom reagować na zmiany danych. DAO zwraca strumień danych, ViewModel je obserwuje i przekształca w stan UI, a interfejs użytkownika automatycznie odświeża widok.
Chain of Responsibility	Behawioralny	AuthInterceptor, ApiLoggingInterceptor, TokenAuthenticator	Żądania HTTP przechodzą przez kolejne elementy łańcucha OkHttpClient. Każdy element może dodać nagłówki, zalogować zapytanie, obsłużyć autoryzację albo przekazać żądanie dalej.
Strategy	Behawioralny	OperationalSyncCoordinator oraz callbacki synchronizacji	Coordinator korzysta z przekazanych z zewnątrz funkcji odpowiadających za sieć, sesję i synchronizację. Dzięki temu sposób działania można zmieniać bez przepisywania głównej logiki koordynatora.
<i>Tabela obejmuje wzorce projektowe zauważalne w projekcie na podstawie struktury kodu oraz katalogu wzorców Refactoring Guru.</i>			

Tabela 8: Wzorce projektowe wykorzystane w aplikacji mobilnej (Mateusz Chimkowski).

7.4 Aplikacja mobilna (Miron Kwiatkowski)

Kategoria GoF	Wzorzec	Lokalizacja	Zastosowanie
Kreacyjny	Singleton	core/database/local_database.dart	Pojedyncza instancja bazy Isar z prywatnym polem statycznym i strażnikiem inicjalizacji.
Kreacyjny	Factory Method	core/network/offline_queue_manager.dart	QueueOperationFactory tworzy wstępnie skonfigurowane obiekty QueuedOperation z UUID i znacznikiem czasu.
Kreacyjny	Abstract Factory	core/network/auth_token_provider.dart	Interfejs AuthTokenProvider i implementacja AuthTokenProviderImpl umożliwia podmiannę w testach.
Strukturalny	Facade	core/network/api_client.dart	Uproszczony interfejs do operacji HTTP (get(), post(), put(), patch(), delete()).
Behawioralny	Strategy	data/repositories/waiter_repository_impl.dart	Przełączanie między API a kolejką offline w zależności od stanu połączenia.
Behawioralny	Observer	core/websocket/data/services/websocket_service.dart	StreamController .broadcast() rozsyła zdarzenia do wielu subskrybentów.
Behawioralny	Strategy	core/websocket/data/handlers/websocket_handlers.dart	Interfejs WSEventHandler<T> i dedykowane handlers dla tematów STOMP.
Behawioralny	Command	domain/usecases/	Każdy przypadek użycia jako osobna klasa z metodą call().
Behawioralny	State	presentation/cubits/	Zarządzanie stanem UI za pomocą Cubit/BLoC.
Behawioralny	Chain of Responsibility	core/network/auth_interceptor.dart	Kolejkowanie żądań podczas odświeżania tokena.

Tabela 9: Wzorce projektowe wykorzystane w aplikacji mobilnej (Miron Kwiatkowski).

7.5 Aplikacja Desktopowa

Kategoria	Wzorzec	Użyty	Gdzie / uwaga
Kreacyjne	Singleton	Tak	Rejestracja usług jako singletony w kontenerze DI: <code>App.xaml.cs</code> , m.in. <code>IRestaurantWebSocketService</code> , <code>IRealtimeUpdateService</code> , <code>IJwtAuthorizationService</code> , <code>INavigationService</code> .
Kreacyjne	Factory Method	Tak	<code>QuiLaCarneDbContextFactory.cs</code> : metoda <code>CreateDbContext()</code> tworzy i konfiguruje instancję <code>QuiLaCarneDbContext</code> .
Strukturalne	Facade	Tak	<code>NavigationService.cs</code> upraszcza otwieranie okien i widoków aplikacji. <code>SyncService.cs</code> udostępnia jedną warstwę do synchronizacji wielu typów danych.
Strukturalne	Adapter	Tak	<code>DpapiStringConverter.cs</code> adaptuje szyfrowanie i deszyfrowanie DPAPI do mechanizmu <code>ValueConverter</code> używanego przez Entity Framework Core.
Behawioralne	Command	Tak	ViewModele wykorzystują <code>[RelayCommand]</code> z <code>CommunityToolkit.Mvvm</code> , np. <code>LoginViewModel.cs</code> , <code>MenuViewModel.cs</code> , <code>ManagerViewModel.cs</code> .
Behawioralne	Observer	Tak	<code>RestaurantWebSocketService.cs</code> i <code>RealtimeUpdateService.cs</code> publikują zdarzenia przez <code>EventHandler</code> . ViewModele subskrybują te zdarzenia i reagują na zmiany danych.
Behawioralne	Strategy	Tak	<code>IJwtAuthorizationService</code> oddziela algorytm sprawdzania uprawnień od logiki logowania. Obecnie istnieje jedna implementacja: <code>JwtAuthorizationService.cs</code> , ale może zostać wymieniona bez zmian w ViewModelu.

Tabela 10: Tabela użytych wzorców projektowych w aplikacji desktopowej

Uwaga metodologiczna: za *użyty* uznano wzorce widoczne bezpośrednio w kodzie aplikacji desktopowej oraz wzorce realizowane przez używane mechanizmy frameworków, jeżeli są faktycznie wykorzystywane w projekcie.

8 Wnioski projektowe

8.1 Mateusz Bogacz-Drewniak

- **Zaawansowana obsługa protokołu WebSocket:** Implementacja komunikacji dwukierunkowej wymagała zmiany podejścia z prostego mechanizmu typu *ping-pong* na pełnowymiarową architekturę sterowaną zdarzeniami. Zaprojektowano system, w którym każda wiadomość niesie za sobą kompletny ładunek danych (*Payload*) w postaci dedykowanego obiektu DTO oraz jednoznaczłą informację o typie akcji.
- **Adaptacja do nowego środowiska programistycznego:** Analiza frameworka Spring Boot pozwoliła na zaobserwowanie analogii oraz kluczowych różnic w stosunku do dotychczas używanych technologii webowych. Praca z nowym ekosystemem rozwinęła umiejętność szybkiej adaptacji i zrozumienia odmiennych paradygmatów konfiguracyjnych oraz architektonicznych.
- **Aspekty bezpieczeństwa aplikacji webowych:** Istotną część projektu poświęcono zagadnieniom z zakresu *Web Security*. Proces ten pozwolił na znaczne poszerzenie wiedzy teoretycznej i praktycznej w obszarze zabezpieczania punktów końcowych API oraz autoryzacji użytkowników.
- **Synergia pomiędzy Clean Architecture a Clean Code:** Realizacja projektu pozwoliła na przewartościowanie podejścia do strukturyzacji kodu. Dotychczasowe skupienie na czystości samej architektury (np. zasada SRP, struktura katalogów) zostało uzupełnione o praktyczne zastosowanie zasad *Clean Code*. Analiza własnych, wcześniejszych błędów programistycznych wykazała, jak kluczowa dla utrzymania systemów jest czytelność kodu na poziomie pojedynczych funkcji i klas.
- **Praktyczna implementacja wzorców projektowych:** Praca nad aplikacją pozwoliła na głębokie zrozumienie celowości stosowania wzorców projektowych. Ich świadome wykorzystanie realnie przełożyło się na elastyczność kodu, łatwość jego rozbudowy oraz uproszczenie procesu implementacji złożonych logik biznesowych.

8.2 Paweł Kruk

- **Wyzwania integracyjne i wdrożeniowe:** Jednym z najbardziej wymagających etapów tworzenia aplikacji okazało się finalne wdrożenie oraz połączenie warstwy frontendowej (React) z serwerem (Java Spring Boot). Złożoność rozproszonej architektury wymagała dużej koordynacji, szczególnie przy synchronizacji mechanizmów autoryzacji, obsługi sesji oraz ujednolicaniu polityki bezpieczeństwa.
- **Scentralizowane zarządzanie stanem i API:** Aby sprawnie zarządzać komunikacją sieciową i uniknąć powielania kodu, zaimplementowano scentralizowane zarządzanie stanem oraz dedykowany mechanizm do obsługi zapytań. Stworzenie jednego, spójnego punktu odpowiedzialnego za dołączanie tokenów i płynną obsługę błędów znacząco ustabilizowało działanie systemu w środowisku produkcyjnym.

- **Optymalizacja dzięki protokołowi WebSocket:** Kluczowym krokiem było zastosowanie architektury sterowanej zdarzeniami. Odejście od klasycznego odpytywania serwera na rzecz komunikacji w czasie rzeczywistym nie tylko zoptymalizowało zużycie zasobów sieciowych, ale przede wszystkim zapewniło błyskawiczną aktualizację interfejsu (np. w przypadku zmian dostępności potraw w menu).
- **Programowanie defensywne i odporność systemu:** Doświadczenia z wdrożenia uwypukliły wagę podejścia defensywnego. Aplikację zaprojektowano tak, aby problemy z modułami pobocznymi (np. chwilowy brak dostępu do słowników z filtrami) nie blokowały jej głównego działania. Dzięki temu krytyczne ścieżki, takie jak przeglądanie oferty czy składanie rezerwacji, pozostają w pełni użyteczne nawet w obliczu przejściowych trudności komunikacyjnych.

8.3 Mateusz Chimkowski

- **Obsługa statusów**
Najtrudniejszym elementem projektu okazała się obsługa statusów stolików, rezerwacji i zamówień. Status nie jest tylko jedną wartością w bazie, bo wpływa na wiele części aplikacji. Trzeba było pilnować, żeby zmiana była poprawnie widoczna w interfejsie, zapisana lokalnie i zsynchronizowana z API.
- **Tryb offline-first**
Projekt pokazał, że aplikacja działająca offline wymaga innego podejścia niż zwykła aplikacja online. Lokalna baza danych musi być pełnoprawnym źródłem danych, a nie tylko prostym cache. Dzięki temu kelner może korzystać z aplikacji nawet przy problemach z internetem.
- **Planowanie synchronizacji**
Synchronizację danych warto planować od samego początku projektu. Dodawanie jej później powoduje problemy z identyfikatorami, kolejnością aktualizacji danych i dopasowaniem modeli lokalnych do danych z API.
- **Podział aplikacji na warstwy**
Przy większym projekcie bardzo pomaga podział kodu na konkretne warstwy, takie jak baza danych, API, repozytoria, ViewModele i ekrany. Dzięki temu łatwiej znaleźć miejsce odpowiedzialne za dany problem i nie mieszać logiki interfejsu z logiką danych.
- **Kotlin i Gradle**
Praca z Kotlinem i Gradle była jednym z mniej przyjemnych elementów projektu. Wynikało to głównie z mojej niechęci do ekosystemu Javy oraz sposobu konfiguracji projektów Androidowych. Szczególnie problematyczne były zależności, wersje bibliotek i konfiguracja Gradle.
- **Testowanie aplikacji mobilnej**
Testowanie aplikacji mobilnej jest bardziej złożone niż testowanie prostych aplikacji desktopowych. Trzeba sprawdzać nie tylko działanie ekranów, ale też lokalną bazę danych, połączenie z API, brak internetu, uprawnienia systemowe i powiadomienia.

- **Zabezpieczenia jako część architektury**

Zabezpieczenia powinny być uwzględnione już na etapie projektowania aplikacji. Szyfrowanie bazy, bezpieczne przechowywanie tokenów i wykorzystanie Android Keystore wpływają na sposób logowania, przechowywania sesji oraz uruchamiania bazy danych.

- **Integracja z API**

Aplikacja mobilna była mocno zależna od struktury API. Każdy endpoint, format odpowiedzi i sposób identyfikacji danych miał wpływ na modele DTO, encje lokalne i synchronizację. Pokazało to, jak ważna jest dobra dokumentacja API.

- **Długie sesje pracy**

Bardzo długie sesje pracy pozwalały szybko rozwijać aplikację, ale z czasem utrudniały debugowanie i podejmowanie dobrych decyzji. Po wielu godzinach łatwo skupić się na naprawianiu objawów problemu zamiast znaleźć jego faktyczną przyczynę.

- **Spójność całej aplikacji**

Projekt pokazał, że w większej aplikacji najtrudniejsze nie jest napisanie pojedynczego ekranu czy funkcji. Największym wyzwaniem jest utrzymanie spójności między interfejsem, bazą danych, API, statusami i synchronizacją.

8.4 Szymon Mikołajek

- **Projektowanie interfejsu w dużej skali:** Praca nad warstwą prezentacji w rozbudowanym projekcie pozwoliła na zmierzenie się z wyzwaniami dotyczącymi wdrażania założeń graficznych oraz utrzymania spójności wizualnej (*UI consistency*). Doświadczenie to przełożyło się na głębsze zrozumienie zasad planowania struktury widoków i organizacji komponentów, co zapewniło wysoką czytelność oraz ergonomię użytkownika aplikacji (*UX*).

- **Adaptacja frameworka WPF w aplikacjach desktopowych:** Wykorzystanie technologii *Windows Presentation Foundation* (WPF) umożliwiło rozwinięcie kompetencji w zakresie budowania aplikacji natywnych. Praktyka ta pozwoliła na opanowanie zaawansowanych mechanizmów organizacji widoków, deklaratywnego stylowania elementów oraz tworzenia przejrzystych struktur interfejsu użytkownika.

- **Mechanizmy bezpieczeństwa i synchronizacji danych:** Istotnym elementem rozwoju było zgłębienie zagadnień związanych z ochroną informacji i synchronizacją stanów w systemie. Zapoznanie się z podstawami zarządzania uprawnieniami użytkowników, zabezpieczaniem danych lokalnych oraz bezpieczną wymianą komunikatów między modułami poszerzyło perspektywę w obszarze projektowania nowoczesnych i bezpiecznych systemów.

- **Zastosowanie wzorców projektowych w optymalizacji kodu:** Analiza i wdrożenie sprawdzonych rozwiązań architektonicznych wpłynęły bezpośrednio na podniesienie jakości oraz wydajności tworzonego oprogramowania. Szczególne skupienie na wzorcu kreatywnym *Prototype* (Prototyp) pozwoliło na zrozumienie technik elastycznej organizacji kodu, co znacząco ułatwia jego późniejszą czytelność i podatność na rozbudowę.

8.5 Miron Kwiatkowski

- **Modelowanie stanów encji jako automatów skończonych**

Statusy stolików, rezerwacji i zamówień nie funkcjonują w izolacji — tworzą graf przejść, w którym każda zmiana wpływa na prezentację (kolorowanie kart w `TablesPage`), dostępne akcje (operacje dozwolone tylko w określonych stanach) oraz strategię synchronizacji (API vs. kolejka offline). Hermetyzacja predykatów stanu w encjach (`isAvailable`, `isReserved`, `requiresCleaning`) ograniczyła rozproszenie logiki, jednak najtrudniejszym przypadkiem pozostały konflikty wynikające z równoczesnych modyfikacji z wielu urządzeń.

- **Kolejka operacji jako fundament trybu offline**

Praca bez połączenia sieciowego wymagała zastąpienia doraźnych instrukcji warunkowych pełnoprawną kolejką operacji. Każda akcja użytkownika — modyfikacja zamówienia, zmiana statusu stolika, przypisanie kelnera — przekształcana jest w obiekt `QueuedOperation` z unikalnym identyfikatorem, typem, ładunkiem i licznikiem ponowień. Przy inicjalizacji `OfflineQueueManager` analizuje stan sieci i opróżnia kolejkę w kolejności FIFO. Kluczowym wnioskiem jest to, że kolejka musi być projektowana równolegle z repozytorium — luka w repozytorium (brak `enqueue` dla `assignWaiter` czy `markAbsent`) oznacza utratę funkcjonalności offline mimo gotowej infrastruktury kolejki.

- **Wstępne zasilenie cache przy starcie aplikacji**

Lokalna baza Isar nie zawiera danych przed pierwszym uruchomieniem. Bez mechanizmu bootstrapu (`syncAllData`) wywoływanego automatycznie po inicjalizacji kontenera DI użytkownik offline nie widzi żadnych stolików, pozycji menu ani zgłoszeń. Rozwiązaniem jest pobranie kompletnego zestawu danych z API podczas uruchamiania i wypełnienie cache, z bezgłośnym pominięciem gdy sieć jest niedostępna. Pokazuje to, że warstwa cache wymaga strategii wstępnego ładowania — nie można zakładać, że użytkownik kiedykolwiek wcześniej korzystał z aplikacji online.

- **Koszty sztywnego podziału na warstwy**

Architektura warstwowa (Domain, Data, Presentation) ułatwia lokalizację błędów — od razu wiadomo, czy przyczyna leży w DTO, repozytorium, czy Cubicie. Jednak każda nowa funkcjonalność wymaga przejścia przez wszystkie trzy warstwy: nowe pole w API oznacza zmianę w DTO, mapperze, encji, modelu Isar, repozytorium, use case i stanie Cubita. Łańcuch ten jest przewidywalny, ale długi — szczególnie przy operacjach typu CRUD, gdzie logika biznesowa jest minimalna, a ilość klas pośredniczących znaczna. W praktyce warto rozważyć, które fragmenty aplikacji rzeczywiście potrzebują pełnej separacji, a gdzie można zastosować uproszczony przepływ danych.

- **Konsekwencje stosowania generacji kodu**

Narzędzia takie jak `json_serializable` z `build_runner` oraz adnotacje `Isar @collection` generują setki linii kodu w kilka sekund, eliminując ręczne parsowanie JSON i definiowanie schematów. Jednak każda zmiana sygnatury pola w DTO wymaga ponownego uruchomienia `build_runnera`, a konflikty w plikach `.g.dart` mogą blokować kompilację. W codziennej pracy narzut ten okazał się istotny — szczególnie podczas refaktoringu encji, gdzie pojedynczy błąd w adnotacji powoduje kaskadę błędów w kodzie generowanym.

- Testowanie scenariuszy brzegowych zamiast ścieżki podstawowej**
 Testowanie aplikacji kelnerskiej nie ogranicza się do weryfikacji poprawności renderowania ekranów. Wymaga pokrycia przypadków skrajnych: konflikt między operacją offline a zmianą wprowadzoną przez innego użytkownika, wygaśnięcie tokena JWT podczas synchronizacji kolejki 20 operacji, kolizja między aktualizacją WebSocket a lokalnie wykonywaną modyfikacją. Te scenariusze ujawniają luki architektoniczne niewidoczne przy testowaniu pojedynczych Cubitów w izolacji.
- Zarządzanie sesją jako zagadnienie architektoniczne**
 Przechowywanie JWT w `flutter_secure_storage` i wstrzykiwanie go przez `AuthInterceptor` jest prostym mechanizmem — do momentu wygaśnięcia tokena. Wówczas interceptor wstrzymuje wszystkie równoczesne żądania (wykorzystując `Completer`), odświeża token, zapisuje go w storage i ponawia oryginalne żądania. Błąd w tej sekwencji — na przykład wysłanie nieaktualnego tokena po operacji odświeżenia — skutkuje wylogowaniem użytkownika. Mechanizm sesji nie jest dodatkiem, lecz fundamentem warstwy sieciowej.
- Zależność modeli od kontraktu API**
 Struktura endpointów REST bezpośrednio determinuje kształt warstwy danych. Paginacja wymusiła wprowadzenie generycznej klasy `PaginatedResponseDto<T>`, odpowiedzi słownikowe — osobnych DTO dla kategorii dań, alergenów i statusów, a identyfikacja przez tokeny — obecność pola `token` w każdej encji. Zmiana konwencji API pociągnęłaby za sobą modyfikację większości DTO w projekcie. Wskazuje to na konieczność ustabilizowania kontraktu API przed rozpoczęciem implementacji warstwy danych.
- Integracja komunikacji czasu rzeczywistego**
 STOMP przez `stomp_dart_client` został zaimplementowany w pełnej strukturze trójwarstwowej: modele domenowe (`WebSocketEvent<T>`), serwis ze strumieniami broadcast (`WebSocketService`), 15 handlerów (`WSEventHandler<T>`) oraz dedykowany Cubit (`WebSocketCubit`). Mimo kompletności infrastruktury handlery pozostały niepodłączone do aktualizacji stanu Cubitów domenowych — rejestrują zdarzenia wyłącznie w konsoli. Dowodzi to, że najtrudniejszym etapem jest nie odebranie zdarzenia, lecz jego integracja z resztą architektury przy uwzględnieniu potencjalnych konfliktów z operacjami w toku i synchronizacją offline.
- Problem wielu reprezentacji tego samego konceptu**
 Największym wyzwaniem architektonicznym nie była implementacja pojedynczego ekranu czy endpointu, lecz utrzymanie spójności między wieloma reprezentacjami tego samego bytu. Status stolika istnieje równocześnie w encji domenowej, DTO z API, modelu Isar, stanie Cubita i zdarzeniu WebSocket. Każda z tych reprezentacji może ulec desynchronizacji — a gdy do tego dojdzie, aplikacja prezentuje użytkownikowi nieprawdziwy stan. Nadrzędnym celem architektury jest zapobieganie temu zjawisku, a nie jedynie realizacja podręcznikowych wzorców.

Spis rysunków

1	Diagram związków encji (ERD) przedstawiający strukturę bazy danych systemu.	11
2	Przykładowy widok dokumentacji backend cz.1.	12
3	Przykładowy widok dokumentacji backend cz.2.	12
4	Przykładowy widok dokumentacji backend cz.3.	13
5	Przykładowy widok z aplikacji mobilnej cz.1	14
6	Przykładowy widok z aplikacji mobilnej cz.2	14
7	Przykładowy widok z aplikacji mobilnej cz.3	15
8	Przykładowy widok z aplikacji mobilnej cz.1 (Miron)	15
9	Przykładowy widok z aplikacji mobilnej cz.2 (Miron)	15
10	Przykładowy widok aplikacji web cz.1.	16
11	Przykładowy widok aplikacji web cz.2.	16
12	Przykładowy widok aplikacji web cz.3.	17
13	Przykładowy widok aplikacji desktop cz.1.	17
14	Przykładowy widok aplikacji desktop cz.2.	18
15	Przykładowy widok aplikacji desktop cz.3.	18

Spis tabel

1	Szczegółowy stack technologiczny backendu wraz z wersjami bibliotek. . . .	6
2	Szczegółowy stack technologiczny aplikacji webowej (frontendu) wraz z wersjami.	7
3	Szczegółowy stack technologiczny aplikacji mobilnej.	8
4	Szczegółowy stack technologiczny aplikacji mobilnej (Moduł Kelnera). . . .	9
5	Szczegółowy stack technologiczny aplikacji desktopowej.	10
6	Tabela użytych wzorców projektowych na backendzie	27
7	Wzorce projektowe w warstwie frontendu (React).	28
8	Wzorce projektowe wykorzystane w aplikacji mobilnej (Mateusz Chimkowski).	29
9	Wzorce projektowe wykorzystane w aplikacji mobilnej (Miron Kwiatkowski).	30
10	Tabela użytych wzorców projektowych w aplikacji desktopowej	31

Spis fragmentów kodu

1	Klonowanie repozytorium backend	19
2	Przygotowanie pliku .env backend	19
3	Przykładowy .env dla backend	20
4	Uruchomienie lokalne backend	20
5	Uruchomienie kreatora admina backend	21
6	Restart kontenera backend	21
7	Uruchomienie testów backend	21
8	Konfiguracja zmiennych środowiskowych w Render	25